Parallel 2D Heat Diffusion Simulation in MPI

Gautam Rao
EID: gr25734
TACC Username: gautamrao
Email:gautam.r.rao@gmail.com

April 30, 2026

1 Introduction

Heat diffusion is a physical process that describes how thermal energy spreads over time due to temperature differences. The governing equation is the two-dimensional heat equation. This equation models the flow of heat between regions:

$$\frac{\partial T}{\partial t} = \kappa \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right) \tag{1}$$

where $T(x, y, t)$ is the temperature, t is time, x and y are spatial coordinates, and κ is the thermal diffusivity. In this project we are going to be using a Forward Time Centered Difference (FTCS) [1] scheme where the time derivative is approximated using a forward difference and the spatial derivatives are approximated using a central difference. The reason for this is that since this equation is continuous in both time and space it cannot be solved without the discretization. In this paper [1], it gives how to solve the FTCS in 1D, and in this project, it is extended to 2D through applying the approximations in the x and y directions as seen below:

The continuous temperature field is then represented on a discrete grid:

$$T_{i,j}^n \approx T(x_i, y_j, t_n)$$

Time derivative is approximated using a forward difference:

$$\frac{\partial T}{\partial t} \approx \frac{T_{i,j}^{n+1} - T_{i,j}^n}{\Delta t}$$

Second derivative in the x-direction is approximated using a centered difference:

$$\frac{\partial^2 T}{\partial x^2} \approx \frac{T_{i+1,j}^n - 2T_{i,j}^n + T_{i-1,j}^n}{\Delta x^2}$$

Second derivative in the y-direction is done the same way:

$$\frac{\partial^2 T}{\partial y^2} \approx \frac{T_{i,j+1}^n - 2T_{i,j}^n + T_{i,j-1}^n}{\Delta y^2}$$

Substitute finite-difference approximations into the heat equation:

$$\frac{T_{i,j}^{n+1} - T_{i,j}^n}{\Delta t} = \kappa \left(\frac{T_{i+1,j}^n - 2T_{i,j}^n + T_{i-1,j}^n}{\Delta x^2} + \frac{T_{i,j+1}^n - 2T_{i,j}^n + T_{i,j-1}^n}{\Delta y^2} \right)$$

Equal grid spacing is assumed in x and y directions:

$$\Delta x = \Delta y = h$$

This allows the two spatial terms to be combined into a single five-point update pattern:

$$\frac{T_{i,j}^{n+1} - T_{i,j}^{n}}{\Delta t} = \frac{\kappa}{h^2}\left(T_{i+1,j}^{n} + T_{i-1,j}^{n} + T_{i,j+1}^{n} + T_{i,j-1}^{n} - 4T_{i,j}^{n}\right)$$

Solving for the next timestep gives the explicit update equation:

$$T_{i,j}^{n+1} = T_{i,j}^{n} + \frac{\kappa \Delta t}{h^2}\left(T_{i+1,j}^{n} + T_{i-1,j}^{n} + T_{i,j+1}^{n} + T_{i,j-1}^{n} - 4T_{i,j}^{n}\right)$$

The constants are grouped into one parameter, α:

$$\alpha = \frac{\kappa \Delta t}{h^2}$$

This produces the update equation used in the code:

$$T_{i,j}^{n+1} = T_{i,j}^{n} + \alpha\left(T_{i-1,j}^{n} + T_{i+1,j}^{n} + T_{i,j-1}^{n} + T_{i,j+1}^{n} - 4T_{i,j}^{n}\right)$$

where (i, j) represents a grid point and n represents the timestep. The parameter α combines the effects of thermal diffusivity, timestep size, and grid spacing. The discrete form allows for the temperature values to be updated locally at each timestep using only the values of neighboring grid points. The global grid is then decomposed into subdomains distributed across multiple MPI processes, where each process updates its local region and exchanges boundary data with its neighbors. These simulations would have real world applications in diffusion models, electronic devices, in material science to study heat processes on material and in other industries as well.

2 Parallelization Strategy

The global grid is decomposed into smaller subregions, with each MPI process responsible for a portion of the domain. MPI was chosen because:

- The data is distributed across processes
- Neighbor communication is required at each timestep

A 2D Cartesian communicator is used to arrange processes logically in a grid. This allows each process to easily determine its neighbors (north, south, east, west) using `MPI_Cart_shift`. To compute updates near boundaries, each process uses exchange buffers. These store boundary values are received from neighboring processes. Communication is performed using `MPI_Sendrecv`, ensuring that each process exchanges edge data before updating its grid.

2.1 Code Example

2.1.1 2D Cartesian Decomposition and Communication

A key component of the parallelization strategy is the use of a 2D Cartesian communicator. This allows MPI processes to be arranged logically in a grid that mirrors the structure of the physical problem domain. Each process is responsible for a subregion of the global grid.

```
int dims[2] = {0, 0};
MPI_Dims_create(nprocs, 2, dims);

int periods[2] = {0, 0};
MPI_Comm cart_comm;
MPI_Cart_create(MPI_COMM_WORLD, 2, dims, periods, 1, &cart_comm);
```

This code determines an optimal 2D grid layout for the processes and creates a Cartesian communicator. The MPI library automatically assigns a grid shape (e.g., 2×2 or 4×2) based on the number of processes. This allows for an evenly partitioned domain.

2.1.2 Process Coordinates and Neighbor Identification

```
int coords[2];
MPI_Cart_coords(cart_comm, cart_rank, 2, coords);

int north, south, west, east;
MPI_Cart_shift(cart_comm, 0, 1, &north, &south);
MPI_Cart_shift(cart_comm, 1, 1, &west, &east);
```

Each process obtains its position in the grid using `MPI_Cart_coords`. Neighboring processes are identified using `MPI_Cart_shift`, which provides the ranks of adjacent processes in each direction. This allows each process to communicate only with its immediate neighbors, which is essential for this computation.

2.1.3 mdspan-Based Grid Representation

```
struct mdspan2d {
  double* data;
  int rows, cols;

  double& operator()(int i, int j) {
    return data[i * cols + j];
  }
};
```

The grid is stored as a one-dimensional array for memory efficiency, but accessed using a custom mdspan interface. This allows for efficient access to grid values and matches the structure of the heat diffusion equation.

2.1.4 Exchange Buffers for Boundary Communication

```
std::vector<double> current_data((local_nx + 2) * (local_ny + 2), 0.0);
mdspan2d current{current_data.data(), local_nx + 2, local_ny + 2};
```

The use of exchange buffers is essential in this method as each grid point depends on values from its neighbors. Each process allocates additional exchange buffers to store data received from these neighboring processes. At every timestep, processes exchange edge values via MPI communication,

populating these buffers. This allows all computations to be performed locally after communication, eliminating the need for further data exchange during the update step. This, in turn, improves our parallel efficiency.

2.1.5 Row Communication (North-South Exchange)

```
MPI_Sendrecv(&current(1,1), local_ny, MPI_DOUBLE, north, 0,
             &current(local_nx+1,1), local_ny, MPI_DOUBLE, south, 0,
             cart_comm, MPI_STATUS_IGNORE);
```

This operation exchanges boundary rows between neighboring processes. The top row is sent to the north neighbor, while data from the south neighbor is received into the corresponding exchange buffer. The use of `MPI_Sendrecv` enables simultaneous send and receive operations, preventing deadlock and ensuring consistent data exchange.

2.1.6 Column Communication (East-West Exchange)

```
MPI_Sendrecv(&current(1,1), 1, column_type, west, 2,
             &current(1, local_ny + 1), 1, column_type, east, 2,
             cart_comm, MPI_STATUS_IGNORE);

MPI_Sendrecv(&current(1, local_ny), 1, column_type, east, 3,
             &current(1, 0), 1, column_type, west, 3,
             cart_comm, MPI_STATUS_IGNORE);
```

These operations also exchange boundary columns with neighboring processes, this time in the east and west directions. Since columns are not contiguous in memory, a custom MPI datatype (`column_type`) is used to represent column data. Received values are stored in exchange buffers, enabling each process to access neighboring boundary data locally during computation.

2.1.7 Heat Diffusion Update

```
next(i,j) = current(i,j) + alpha * (
    current(i-1,j) + current(i+1,j) +
    current(i,j-1) + current(i,j+1) -
    4.0 * current(i,j)
);
```

After exchanging boundary data, each process updates its local grid using a finite-difference stencil. The computation uses only local values and data stored in the exchange buffers, allowing all updates to occur independently without further communication.

3 Experimental Setup

To evaluate the performance of the parallel heat diffusion simulation, experiments were conducted using varying numbers of MPI processes, as well as different timestep configurations. The simulation uses an initial condition consisting of a 20×20 region at the center of the grid set to a temperature of 100 units, while all other cells are initialized to 0. The primary configuration used a 500×500,

1000×1000, or a 5000×5000 grid with 2000, 5000 time steps. The number of processes was varied across 1, 2, 4, and 8 processes to see the effect of parallelization on runtime. The following commands were used:

- `ibrun -n 1 ./heat_distribution <global_nx> <global_ny> <steps>`
- `ibrun -n 2 ./heat_distribution <global_nx> <global_ny> <steps>`
- `ibrun -n 4 ./heat_distribution <global_nx> <global_ny> <steps>`
- `ibrun -n 8 ./heat_distribution <global_nx> <global_ny> <steps>`

Execution time was measured using `MPI_Wtime()`, which records the time required to complete the simulation. The input parameters represent:

- Grid size in the x-direction ($global\_nx$)
- Grid size in the y-direction ($global\_ny$)
- Number of time steps ($steps$)

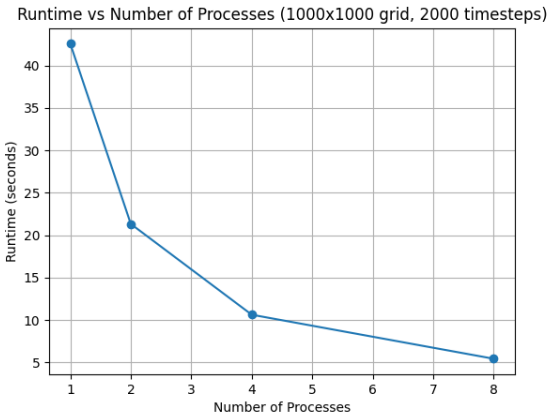
Together, these parameters determine the total workload being put on the simulation.

4 Results and Observations

4.1 Results for 2000 Timesteps

Processes	Runtime (s)	Total Heat
1	42.58	40000
2	21.34	40000
4	10.64	40000
8	5.45	40000

Runtime table



Runtime scaling graph

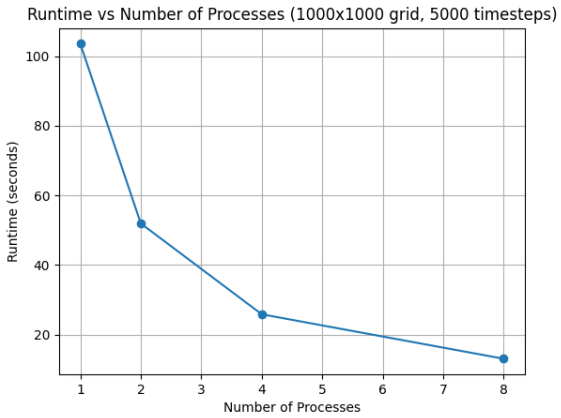
Figure 1: Runtime and scaling for a 1000×1000 grid with 2000 timesteps

The results show that increasing the number of processes significantly reduces runtime, indicating effective distribution of the workload. The decrease in runtime is close to proportional, demonstrating efficient parallel performance. The total heat remains constant across all runs. There are tiny differences in linear performance that may be caused due to communication overhead.

4.2 Results for 5000 Timesteps

Processes	Runtime (s)	Total Heat
1	103.579	40000
2	51.9507	40000
4	25.8538	40000
8	13.08	40000

Runtime table



Runtime scaling graph

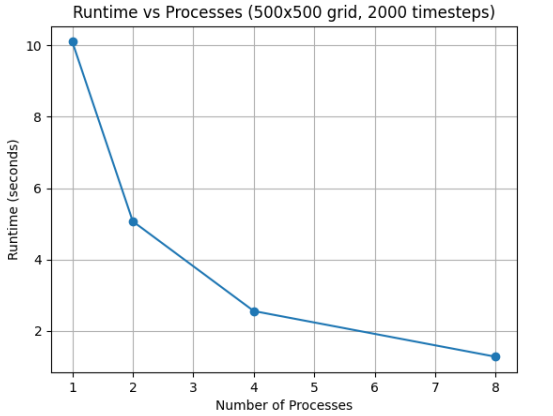
Figure 2: Runtime and scaling for a 1000×1000 grid with 5000 timesteps

We see that by increasing the number of time steps, the runtime increases by a factor of almost 2.5 as compared to the previous run with 2000 time steps. Even with the increased workload, the program still showed constant heat during the run. The scaling behavior remains similar, which shows the parallel implementation is working under a heavier workload.

4.3 Results for 500×500 Grid

Processes	Runtime (s)	Total Heat
1	10.1031	40000
2	5.07009	40000
4	2.5566	40000
8	1.27544	40000

Runtime table



Runtime scaling graph

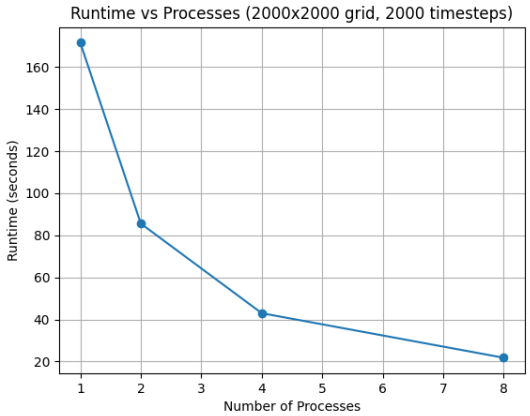
Figure 3: Runtime and scaling for a 500×500 grid with 2000 timesteps

In this trial, the grid size was reduced from a 1000×1000 grid to 500×500. The runtime drops from from 10.1031 seconds with 1 process to 1.27544 seconds with 8 processes, which shows that the work is being divided evenly among the processes. The heat remains constant and is conserved across each of the runs. The trend from the previous trials, where the runtime reduced with more processes, was seen here.

4.4 Results for 2000×2000 Grid

Processes	Runtime (s)	Total Heat
1	171.616	40000
2	85.598	40000
4	42.9541	40000
8	21.7607	40000

Runtime table



Runtime scaling graph

Figure 4: Runtime and scaling for a 2000×2000 grid for 2000 timestep

In this trial, we use a 2000×2000 grid for 2000 time steps, which is a larger grid that, in turn, results in more computational work per time step. This is supported by the results where for 1 process it took 171.616s for this grid as compared to 10.1031s for the 500×500 grid in 4.3. It also shows efficient parallel performance as the runtime goes from 171.616 seconds with 1 process to 21.7607 seconds with 8 processes, demonstrating efficient parallel performance. The total heat remains constant across all runs, which shows that it preserves heat while redistributing it throughout the domain.

4.5 Visualization of 2D heat diffusion on a small grid

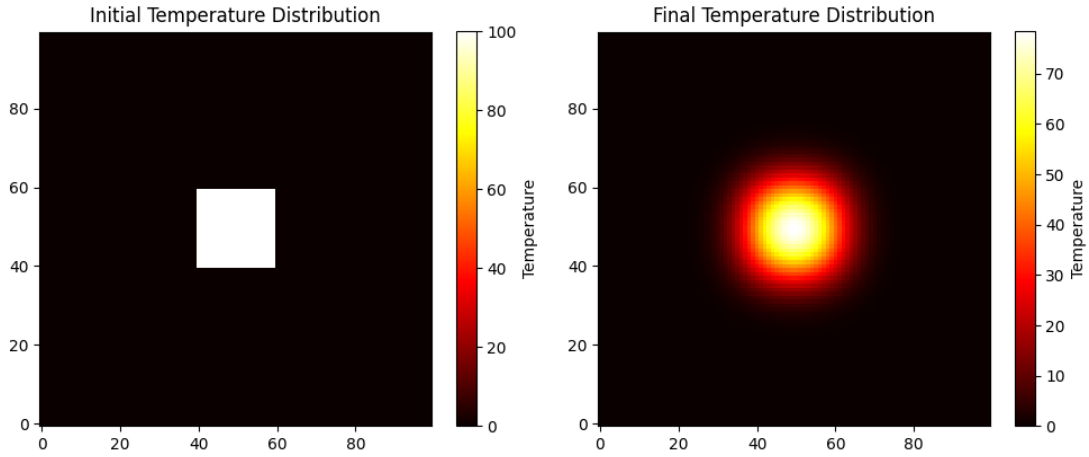


Figure 5: Visualization of heat diffusion on a 2D grid

This graph was generated to portray diffusion over a smaller grid to see if it is working properly. The initial condition is 100 temperature units placed at the center 20×20 portion of the grid. We see over time that diffusion does occur, and the temperature units start to diffuse over the grid.

4.6 Summary of Results

The results demonstrate that both grid size and number of time steps affect the computational cost, while parallelization reduces runtime. As the number of processes increases, the workload is evenly distributed across subregions of the grid, reducing runtime. We see that the total heat remains the same throughout, indicating that the code is preserving heat and distributing it correctly. This is seen to also to also satisfy modeling heat diffusion on a 2D distributed grid, as it shows that the heat is spreading across the grid. Overall, the results demonstrate that the program successfully achieves both accurate physical modeling of heat diffusion and efficient parallel performance using MPI.

5 Performance Analysis

The results show that increasing the number of processes significantly reduces runtime, indicating effective workload distribution. Performance is also seen to improve linearly. Increasing grid sizes and timestep also increases the computational power needed to perform these calculations, and for higher grid sizes and time steps, we see slight drop offs in efficiency and increased run times.

5.1 Future improvements

- Reducing communication overhead through algorithmic refinements
- Improving memory access patterns to enhance cache performance

6 Conclusion

This project demonstrates the successful parallelization of a 2D heat diffusion simulation using MPI. The domain decomposition strategy allows efficient distribution of work across processes, while exchange buffers used for communication of boundary updates. In addition to performance, the simulation correctly models the physical behavior of heat diffusion. The finite-difference update causes heat to spread from a localized high temperature region outward over time, while the total heat remains constant across all runs. This confirms that it conserves physical properties and accurately represents the underlying heat equation.

7 Bibliography

References

[1] "Finite Difference Methods for the Heat Equation (FTCS Scheme),"
Available: `https://webspace.science.uu.nl/~zegel101/MOLMODWISK/FDheat2.pdf`

[2] Victor Eijkhout, *MPI Course Notes*, Texas Advanced Computing Center (TACC).

[3] Victor Eijkhout, *The Art of High Performance Computing*